

[CI/CD](#) конвейер (иногда называемый конвейером развертывания) — краеугольный камень DevOps. Согласно [Gitlab](#):

CI/CD автоматизирует большую часть или всё ручное вмешательство человека, традиционно необходимое для доставки нового кода из коммита в production. С CI/CD конвейером команды разработки могут вносить изменения в код, которые затем автоматически тестируются и вытаскиваются для доставки и развертывания. Правильно настроенный CI/CD минимизирует простои и ускоряет выпуск кода.

Давайте теперь посмотрим, как можно настроить конвейер развертывания, который можно использовать для автоматического развертывания контейнеризованного программного обеспечения на *любую* машину. Так что каждый раз, когда вы коммитите код на вашей машине, конвейер собирает образ и запускает его на сервере.

Поскольку мы не можем предполагать, что у каждого есть доступ к своему собственному серверу, мы продемонстрируем конвейер, используя *локальную машину* как цель разработки, но точно такие же шаги можно использовать для виртуальной машины в облаке (такой как предоставленная [Hetzner](#)) или даже Raspberry Pi.

Мы будем использовать [GitHub Actions](#) для сборки образа и пуша образа в Docker Hub, а затем использовать проект под названием [Watchtower](#) для автоматической загрузки и перезапуска нового образа на целевой машине.

В качестве примера мы рассмотрим репозиторий <https://github.com/docker-hy/docker-hy.github.io>, то есть материалы этого курса.

Как было сказано, [GitHub Actions](#) используется для реализации первой части конвейера развертывания. [Документация](#) дает следующий обзор:

GitHub Actions — это платформа непрерывной интеграции и непрерывной доставки (CI/CD), которая позволяет автоматизировать ваш конвейер сборки, тестирования и развертывания. Вы можете создавать рабочие процессы, которые собирают и тестируют каждый коммит и каждый pull request в ваш репозиторий, или развертывают объединенные pull requests в production.

[Проект](#) определяет workflow с GitHub Actions, который собирает Docker образ и пушит его в Docker Hub каждый раз, когда код пушится в GitHub репозиторий.

Давайте теперь посмотрим, как выглядит определение workflow. Оно хранится в файле `deploy.yml` внутри директории `.github/workflows`:

```
name: Release DevOps with Docker # Name of the workflow

# On a push to the branch named master
on:
  push:
    branches:
      - master

# Job called build runs on ubuntu-latest
jobs:
  deploy:
    name: Deploy to GitHub Pages
    # we are not interested in this job
```

```

publish-docker-hub:
  name: Publish image to Docker Hub
  runs-on: ubuntu-latest
  steps:
    # Checkout to the repository
    - uses: actions/checkout@v2

    # We need to login so we can later push the image without issues.
    - name: Login to Docker Hub
      uses: docker/login-action@v1
      with:
        username: ${ secrets.DOCKERHUB_USERNAME }
        password: ${ secrets.DOCKERHUB_TOKEN }
    # Builds devopsdockeruh/docker-hy.github.io
    - name: Build and push
      uses: docker/build-push-action@v2
      with:
        push: true
        tags: devopsdockeruh/coursepage:latest

```

[Workflow](#) имеет два [jobs](#), нас сейчас интересует тот, который называется *publish-docker-hub*. Другой job, называемый *deploy*, заботится о развертывании страницы как GitHub page.

Job состоит из серии [steps](#). Каждый шаг — это небольшая операция или *action*, которая делает свою часть целого. Шаги следующие:

- [actions/checkout@v2](#) используется для checkout кода из репозитория
- [docker/login-action@v1](#) используется для входа в Docker Hub
- [docker/build-push-action@v2](#) используется для сборки образа и пуша в Docker Hub

Первый action был одним из готовых actions, которые предоставляет GitHub. Последние два — официальные actions, предлагаемые Docker. См. [здесь](#) для получения дополнительной информации об официальных Docker GitHub Actions.

Перед тем как workflow заработает, два [secrets](#) должны быть добавлены в GitHub репозиторий: `DOCKERHUB_TOKEN` и `DOCKERHUB_USERNAME`. Это делается открытием репозитория в браузере и сначала нажатием *Settings*, затем *Secrets*. `DOCKERHUB_TOKEN` можно создать в Docker Hub из *Account Settings / Security*.

GitHub Actions делают только "первую половину" конвейера развертывания: они гарантируют, что каждый push в GitHub собирается в Docker образ, который затем пушится в Docker Hub.

Вторая половина конвейера развертывания реализована контейнеризованным сервисом под названием [Watchtower](#), который является проектом с открытым исходным кодом, автоматизирующим задачу обновления образов. Watchtower будет проверять источник образа (в данном случае Docker Hub) на наличие изменений в работающих контейнерах. Работающий контейнер будет обновлен и автоматически перезапущен, когда новая версия образа будет запущена в Docker Hub. Watchtower уважает теги, например, контейнер, использующий `ubuntu:22.04`, не будет обновлен, пока не будет выпущена новая версия `ubuntu:22.04`.

:::tip Напоминание о безопасности: Docker Hub, получающий доступ к вашему компьютеру

Обратите внимание, что теперь любой, у кого есть доступ к вашему Docker Hub, также имеет доступ к вашему ПК через это. Если они пушат вредоносное обновление в ваше приложение, Watchtower с

радостью загрузит и запустит обновленную версию.

...

Watchtower можно запустить, например, используя следующий файл Docker Compose:

```
version: "3.8"

services:
  watchtower:
    image: containrrr/watchtower
    environment:
      - WATCHTOWER_POLL_INTERVAL=60 # Poll every 60 seconds
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    container_name: watchtower
```

Нужно быть осторожным при запуске Watchtower с помощью *docker compose up*, так как он будет пытаться обновить **каждый** образ, работающий на машине. [Документация](#) описывает, как это можно предотвратить.

Упражнения 3.1-3.4

:::info Упражнение 3.1: Ваш конвейер

Создайте теперь похожий конвейер развертывания для простого Node.js/Express приложения, которое можно найти [здесь](#).

Либо клонируйте проект, либо скопируйте файлы в свой собственный репозиторий. Настройте похожий конвейер развертывания (или "первую половину") с использованием GitHub Actions, который был только что описан. Убедитесь, что новый образ пушится в Docker Hub каждый раз, когда вы пушите код в GitHub (вы можете, например, изменить сообщение, которое показывает приложение).

Обратите внимание, что есть важное изменение, которое вы должны сделать в приведенной выше конфигурации workflow, ветка должна называться *main*:

```
name: Release Node.js app

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      # ...
```

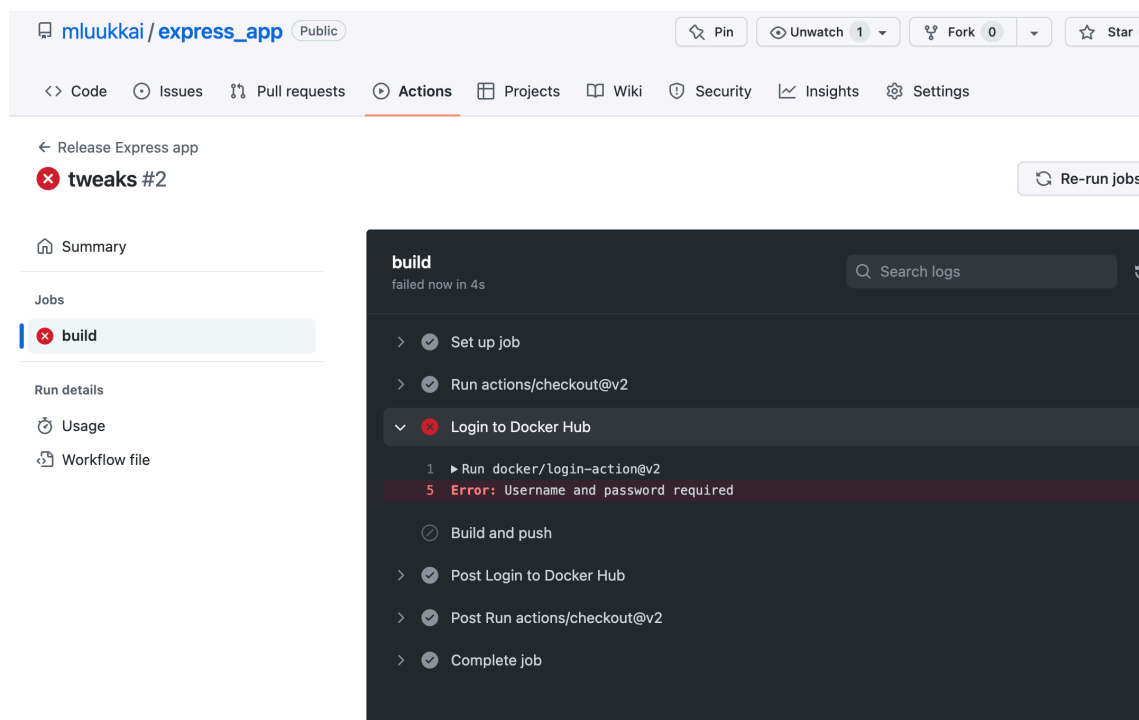
Предыдущий пример все еще использует старое соглашение об именовании GitHub и называет основную ветку *master*.

Некоторые из actions, которые использует приведенный выше пример, немного устарели, так что пройдитесь по документации

- [actions/checkout](#)
- [docker/login-action](#)
- [docker/build-push-action](#)

и используйте самые последние версии в вашем workflow.

Следите за страницей GitHub Actions, чтобы увидеть, что ваш workflow работает:



Также убедитесь из Docker Hub, что ваш образ пушится туда.

Затем запустите ваш образ локально в detached режиме и убедитесь, что вы можете получить к нему доступ с помощью браузера.

Теперь настройте и запустите [Watchtower](#) так, как описано выше.

Вы можете сделать эти два шага в одном с общим файлом Docker Compose.

Теперь ваш конвейер развертывания настроен! Убедитесь, что он работает:

- внесите изменение в ваш код
- закоммитьте и запушьте изменения в GitHub
- подождите некоторое время (время, необходимое GitHub Action для сборки и пуша образа плюс интервал опроса Watchtower)
- перезагрузите браузер, чтобы убедиться, что Watchtower запустил новую версию (то есть, ваши изменения видны)

Отправьте ссылку на репозиторий с конфигурацией.

:::

:::info Упражнение 3.2: Конвейер развертывания в облачный сервис

В [Упражнении 1.16](#) вы развернули контейнеризованное приложение в облачный сервис.

Теперь пришло время улучшить ваше решение, настроив конвейер развертывания для него, так что каждый push в GitHub приводит к новому развертыванию в облачный сервис.

Скорее всего, вы найдете готовый GitHub Action, который делает большую часть тяжелой работы за вас... Google — ваш друг!

Отправьте ссылку на репозиторий с конфигурацией. README репозитория должен содержать ссылку на развернутое приложение.

:::

:::info Упражнение 3.3: Магия скриптов

Создайте скрипт/программу, которая загружает репозиторий с GitHub, собирает Dockerfile, расположенный в корне, а затем публикует его в Docker Hub.

Вы можете использовать любой язык сценариев или программирования для реализации скрипта. Использование [shell script](#) может сделать следующее упражнение немного проще... и не беспокойтесь, если вы раньше не делали shell скрипт, вам не нужно много для этого упражнения, и Google помогает.

Скрипт может быть разработан для использования так, что в качестве первого аргумента он получает GitHub репозиторий, а в качестве второго аргумента — Docker Hub репозиторий. Например, при запуске следующим образом

```
./builder.sh mluukkai/express_app mluukkai/testing
```

скрипт клонирует https://github.com/mluukkai/express_app, собирает образ и пушит его в Docker Hub репозиторий mluukkai/testing

:::

:::info Упражнение 3.4: Сборка образов изнутри контейнера

Как видно из файла Docker Compose, Watchtower использует том для сокета [docker.sock](#) для доступа к Docker даемон хоста из контейнера:

```
services:
watchtower:
  image: containrrr/watchtower
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
# ...
```

На практике это означает, что Watchtower может выполнять команды на Docker так же, как мы можем "командовать" Docker из cli с помощью *docker ps*, *docker run* и т.д.

Мы можем легко использовать тот же трюк в наших собственных скриптах! Так что если мы смонтируем сокет *docker.sock* в контейнер, мы можем использовать команду *docker* внутри контейнера, так же, как мы используем ее в терминале хоста!

Dockerize теперь скрипт, который вы сделали для предыдущего упражнения. Вы можете использовать образы из [этого репозитория](#) для запуска Docker внутри Docker!

Ваш Dockerized может быть запущен так (команда разделена на несколько строк для лучшей читаемости, обратите внимание, что копирование многострочной команды не работает):

```
docker run -e DOCKER_USER=mluukkai \  
-e DOCKER_PWD=password_here \  
-v /var/run/docker.sock:/var/run/docker.sock \  
builder mluukkai/express_app mluukkai/testing
```

Обратите внимание, что теперь учетные данные Docker Hub определены как переменные окружения, так как скрипт должен войти в Docker Hub для пуша.

Отправьте Dockerfile и финальную версию вашего скрипта.

Подсказка: вам, скорее всего, понадобится использовать [ENTRYPOINT](#) в этом Упражнении. См. [Часть 1](#) для получения дополнительной информации.

...